

Roteiro da Atividade Prática

Separabilidade, Suavização, Filtros Não Lineares e Detecção de Bordas por Derivadas de Primeira Ordem

Disciplina: Processamento Digital de Imagens

1 Apresentação da atividade

Antes de iniciar a implementação, releia cuidadosamente os materiais teóricos e os slides das duas últimas aulas. Em particular, revise os conceitos de convolução e correlação espacial. Releia também a Seção 3.5 do livro do Gonzalez (em português), na qual são discutidos os filtros lineares de suavização. Para o estudo da separabilidade do filtro gaussiano, consulte com atenção a seção *Separable Gaussian Filters* em Burger and Burge (2022), p. 96, bem como os slides 9 e 10 da aula de 18/5. Para os filtros de primeira ordem e detecção de bordas, revise os slides da aula passada sobre derivadas discretas e consulte o Capítulo 5 de Burger and Burge (2022), além da discussão de Gonzalez sobre o gradiente, incluindo o procedimento apresentado na p. 469. O filtro de mediana deve ser estudado a partir do exemplo apresentado por Burger and Burge (2022), p. 106, com destaque para a Figura 5.14.

O objetivo não é apenas executar funções prontas de bibliotecas, mas compreender o que está sendo calculado, as imagens processadas e qual é o custo computacional de cada operação. Assim, o Notebook Colab entregue deverá conter o código, as imagens, os gráficos, as tabelas e comentários explicativos em texto. A prática deve ser apresentada como uma pequena narrativa experimental: problema, método, resultado e interpretação.

2 Imagens e Padronização

Para tornar os resultados comparáveis entre os estudantes, as imagens geradas por um procedimento padronizado são fornecidas em anexo. Essa padronização é importante principalmente na análise de complexidade computacional, visto que imagens pequenas podem mascarar as diferenças de desempenho entre a convolução bidimensional direta e a implementação separável.

São fornecidas duas imagens com e sem ruídos em tons de cinza, tipo `uint8`, com alta resolução espacial. Para os testes de desempenho, utilize as resoluções Full HD (1920×1080) e 4K (3840×2160). A imagem 4K é particularmente interessante por possuir aproximadamente quatro vezes mais pixels.

Você também pode usar imagens de sua autoria. Para a detecção de bordas, dê preferência a imagens similares em tamanho e tons de cinza, contendo ele-

mentos geométricos, estruturas com contornos bem definidos, regiões contrastantes, linhas ou objetos que facilitem a interpretação das respostas dos operadores de gradiente. Sempre que comparar métodos diferentes, use exatamente a mesma imagem de entrada, o mesmo ruído (garantindo a mesma semente no gerador pseudoaleatório) e a mesma forma de visualização.

3 Suavização e Separabilidade

A primeira parte da atividade trata da suavização com filtros gaussianos. Nesse processo, o kernel atribui pesos maiores aos pixels próximos do centro da vizinhança e pesos menores aos pixels mais distantes. Entretanto, existe um problema computacional inerente a essa operação. Um filtro gaussiano bidimensional de tamanho $k \times k$, aplicado diretamente sobre uma imagem de dimensão $M \times N$, exige uma quantidade de operações da ordem de

$$T_{2D}(M, N, k) \sim M N k^2, \quad (1)$$

isto é, possui complexidade assintótica $O(MNk^2)$. Isso ocorre porque, para cada pixel da imagem, a operação precisa considerar aproximadamente k^2 amostras da vizinhança. Para kernels pequenos, esse custo não costuma ser um gargalo. Porém, quando k cresce, especialmente em imagens Full HD ou 4K, o tempo de execução pode se tornar proibitivo.

A propriedade fundamental que será explorada aqui é que o filtro gaussiano é *separável*. Isso significa que um filtro gaussiano bidimensional pode ser escrito como o produto externo de dois filtros unidimensionais e implementado como duas convoluções 1D sucessivas: uma aplicada na direção horizontal e outra na direção vertical. Nesse caso, o custo deixa de depender de k^2 e passa a depender de $2k$, isto é,

$$T_{\text{sep}}(M, N, k) \sim 2 M N k, \quad (2)$$

com complexidade assintótica $O(MNk)$. Assim, em vez de aplicar um kernel $k \times k$, aplicam-se dois kernels 1D de tamanho k . A diferença entre k^2 e $2k$ torna-se muito expressiva para kernels grandes. Por exemplo, para $k = 51$, a convolução direta utiliza uma vizinhança com 2601 pesos, enquanto a implementação separável utiliza duas passagens com 51 pesos cada, totalizando 102 pesos por pixel (desconsiderando detalhes de implementação e otimizações internas de hardware).

Nesta parte, utilize a imagem com ruído gaussiano artificial e aplique a suavização de duas formas: primeiro, por convolução bidimensional direta; depois, por convolução separável em duas etapas. Teste diferentes tamanhos de kernel, por exemplo:

$$k \in \{3, 5, 9, 15, 31, 51\}. \quad (3)$$

Se estiver usando a imagem 4K e o tempo de execução for aceitável, você pode também testar kernels maiores, como 71 ou 101. Para cada tamanho de kernel, meça o tempo de execução da convolução direta e da separável. Faça mais de uma repetição de cada medição e apresente o tempo médio (e, se possível, o desvio-padrão). Ao medir o tempo, **não inclua** a leitura de arquivo, a geração de ruído, a exibição de imagens ou a impressão de resultados. Meça rigorosamente apenas a operação de filtragem.

No Notebook Colab, apresente a imagem original, a imagem com ruído gaussiano e exemplos de imagens suavizadas pelos dois métodos. Em seguida, crie uma tabela contendo o tamanho do kernel, o tempo médio da convolução direta, o tempo médio da convolução separável e a razão entre esses tempos. Apresente também um gráfico do tempo de execução em função do tamanho do kernel e outro gráfico ilustrando a tendência teórica das complexidades $O(MNk^2)$ e $O(MNk)$.

Na discussão, explique por que a separabilidade reduz o custo computacional. Comente em quais tamanhos de kernel a diferença ficou mais evidente. Discuta também por que as curvas medidas experimentalmente podem não coincidir exatamente com as curvas teóricas. Otimizações de bibliotecas, cache de memória, tipo de dado, paralelização interna e *overhead* de chamadas de função podem influenciar os tempos medidos. Mesmo assim, a tendência geral esperada é que a implementação separável se torne cada vez mais vantajosa à medida que o tamanho do kernel cresce.

4 Mediana, Sal e Pimenta

Enquanto o ruído gaussiano é bem tratado por filtros lineares de suavização, nem todo ruído tem essa natureza. Um caso clássico é o ruído impulsivo, frequentemente chamado de ruído sal e pimenta. Nesse tipo de degradação, parte dos pixels da imagem é substituída por valores extremos (muito baixos ou muito altos), produzindo pontos pretos e brancos espalhados aleatoriamente.

Filtros lineares, como o de média ou o gaussiano, tendem a espalhar a influência desses valores extremos pela vizinhança. Por isso, eles até suavizam a imagem, mas raramente removem os impulsos de forma eficaz. O filtro de mediana, por sua vez, é não linear e substitui cada pixel pela mediana dos valores em sua vizinhança. Como a mediana é uma estatística robusta (isto é, pouco sensível a *outliers* quando comparada à média), esse filtro costuma ser muito mais eficiente para remover o ruído sal e pimenta.

Nesta parte, tome a imagem degradada por ruído sal e pimenta com densidade de 10%, conforme discu-

tido nos slides. Em seguida, aplique um filtro gaussiano e um filtro de mediana. Se desejar, inclua também um filtro de média para enriquecer a comparação. Para o filtro de mediana, teste janelas de tamanhos:

$$3 \times 3, \quad 5 \times 5, \quad 7 \times 7. \quad (4)$$

No Notebook, apresente a imagem original, a imagem degradada e os resultados obtidos com cada filtro, realizando a comparação visual lado a lado. A discussão deve justificar por que a mediana é mais adequada para esse tipo de ruído. Comente também o que acontece quando se aumenta o tamanho da janela da mediana: a remoção de ruído melhora, mas detalhes finos e pequenas estruturas podem ser suprimidos. Existe um claro compromisso (*trade-off*) entre remoção de ruído e preservação de detalhes. Relacione seus resultados práticos com o exemplo de Burger and Burge (2022), p. 106, especialmente a Figura 5.14.

5 Bordas e Prewitt

Bordas são regiões da imagem onde há variações bruscas de intensidade. Uma forma clássica de detectar essas variações é aproximar derivadas de primeira ordem por máscaras discretas. As máscaras de Prewitt combinam uma operação de derivação em uma direção com uma suavização simples na direção perpendicular.

A interpretação principal é a seguinte: uma máscara estima variações na direção horizontal da imagem, realçando bordas verticais, enquanto a outra estima variações na direção vertical, realçando bordas horizontais (vide slides da última aula).

Use agora imagens com bordas bem definidas. Adicione ruído gaussiano à imagem e aplique as máscaras de Prewitt para calcular as componentes G_x e G_y do gradiente. Em seguida, calcule a magnitude do gradiente por:

$$G = \sqrt{G_x^2 + G_y^2}. \quad (5)$$

Como alternativa, você pode também calcular e apresentar a aproximação:

$$G \approx |G_x| + |G_y|. \quad (6)$$

Apresente no Notebook a imagem original, a imagem com ruído, a resposta G_x , a resposta G_y e a magnitude do gradiente. Se optar por aplicar uma limiarização para obter uma imagem binária de bordas, explique como o limiar foi escolhido (por exemplo, percentil da magnitude, valor fixo empírico ou método automático, como Otsu). Lembre-se de que imagens contendo derivadas podem assumir valores negativos e, por isso, precisam ser normalizadas adequadamente para a visualização correta (por exemplo, mapeando o 0 absoluto para o cinza médio).

A discussão deve focar na diferença visual entre G_x , G_y e a magnitude total do gradiente. Indique quais tipos de bordas são realçados por cada componente e como a presença de ruído interfere na resposta das

derivadas. Relacione seus resultados com os slides da aula sobre derivadas de primeira ordem e com o procedimento discutido em Gonzalez, p. 469. Reflita, por fim, sobre por que derivadas discretas tendem a amplificar ruído de alta frequência e por que as operações de pré-suavização são fundamentais antes da etapa de diferenciação.

6 Bordas e Sobel

Na última parte da prática, você repetirá o processo de detecção de bordas, mas substituindo o operador pelo de Sobel. Assim como o Prewitt, o Sobel estima derivadas de primeira ordem. A diferença sutil reside na ponderação usada na suavização perpendicular à direção da derivada.

O operador de Sobel dá maior peso aos pixels da região central da vizinhança na direção de suavização, aproximando-se do comportamento de uma suavização gaussiana unidimensional. Na prática, isso tende a produzir uma resposta levemente mais robusta ao ruído do que o Prewitt, embora ambos continuem sendo operadores diferenciais e, portanto, sensíveis a variações locais de alta frequência.

Use a **mesma imagem** usada na parte anterior, com o **mesmo nível de ruído gaussiano** (idealmente, com a mesma semente). Aplique as máscaras de Sobel para obter G_x , G_y e a magnitude do gradiente. Em seguida, compare os resultados do Sobel com os do Prewitt. Para garantir que a comparação

seja justa, mantenha rigorosamente a mesma imagem, o mesmo ruído, o mesmo procedimento de normalização e os mesmos critérios de limiarização.

No Notebook, apresente lado a lado: Prewitt G_x versus Sobel G_x ; Prewitt G_y versus Sobel G_y ; além das magnitudes dos dois operadores. Discuta se o Sobel apresentou melhor supressão do ruído e se essa diferença foi visualmente perceptível. Retome a pergunta feita nos slides da última aula (incluindo o exemplo da cabeça de uma zebra) e responda-a agora fundamentado nos seus dados experimentais.

7 Entrega

A entrega principal será um único Notebook Google Colab contendo todos os códigos, figuras, tabelas, gráficos e discussões. O notebook deve estar organizado como um autêntico relatório experimental. Não entregue apenas uma sequência árida de células de código; intercale blocos de texto (Markdown) explicando o que está sendo feito antes do código e interpretando os resultados gerados logo após a execução.

Caso haja dificuldade em documentar adequadamente a discussão no próprio Colab, poderá ser submetida uma apresentação suplementar em formato .pptx. Ressalta-se, porém, que a apresentação não substitui a obrigatoriedade da entrega do notebook com os códigos executáveis e os resultados experimentais.